5 A web app's security begins with filters

This chapter covers

- Working with the filter chain
- Defining custom filters
- Using Spring Security classes that implement the `Filter` interface

In Spring Security, HTTP filters delegate different responsibilities to an HTTP request. Furthermore, they generally manage each responsibility that must be applied to the request. The filters thus form a chain of responsibilities. A filter receives a request, executes its logic, and eventually delegates the request to the next filter in the chain (figure 5.1).

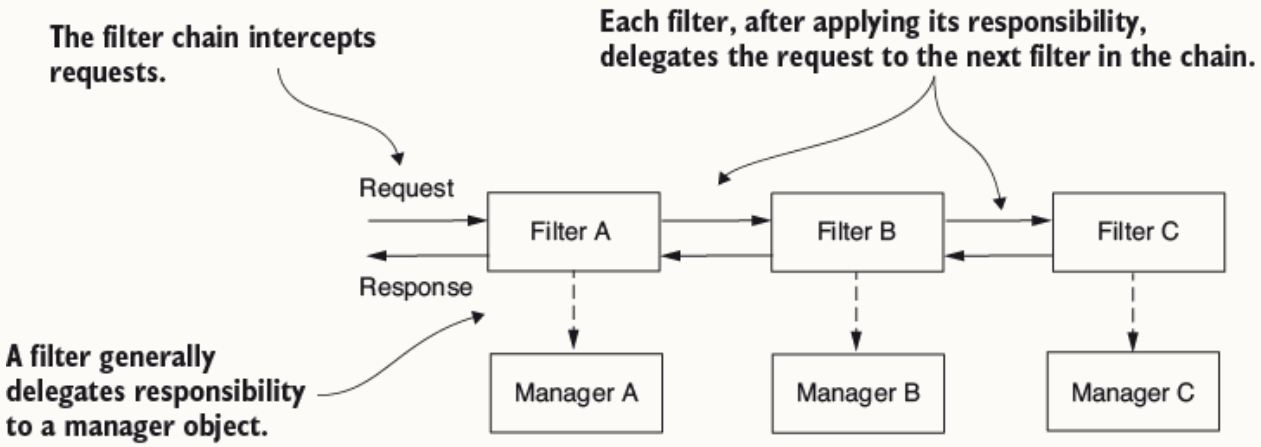


Figure 5.1 The request is passed to the filter chain. Every filter engages a manager to execute particular logic upon the request and then passes it down the line to the subsequent filter in the chain.

Let's take an analogy as an example. When you go to the airport, from entering the terminal to boarding the aircraft, you go through multiple filters (figure 5.2). You first present your ticket, then your passport is verified, and afterward, you go through security. At the airport gate, more filters might be applied. For example, in some cases, right before boarding, your passport and visa are verified again. This is an excellent analogy to the filter chain in Spring Security. In the same way, you customize filters in a filter chain with Spring Security. Spring Security provides filter implementations that you add to the filter chain through customization, but you can also define custom filters.



Figure 5.2 At the airport, you proceed through a series of checkpoints before finally boarding your plane. Similarly, Spring Security implements a sequence of filters that process the HTTP requests the application receives.

This chapter will discuss how to use Spring Security to customize filters that are part of the authentication and authorization architecture in a web app. For example, you might want to augment authentication by adding one more step for the user, such as checking their email address or using a one-time password. You can also add functionality referring to auditing authentication events. You'll find various scenarios where applications use auditing authentication, from debugging purposes to identifying a user's behavior. Today's technology and machine learning algorithms can improve applications, for example, by learning the user's behavior and knowing if somebody hacked their account or impersonated the user.

Knowing how to customize the HTTP filter chain of responsibilities is a valuable skill. In practice, applications come with various requirements where default configurations no longer work. You'll need to add or replace existing components of the chain. With the default implementation, you use the HTTP Basic authentication method, which allows

you to rely on a username and password. However, in practical scenarios, there are plenty of situations in which you'll need more. Maybe you'll need to implement a different strategy for authentication, notify an external system about an authorization event, or log a successful or failed authentication that's used later in tracing and auditing (figure 5.3). Whatever your scenario is, Spring Security offers you the flexibility of modeling the filter chain precisely as you need it.

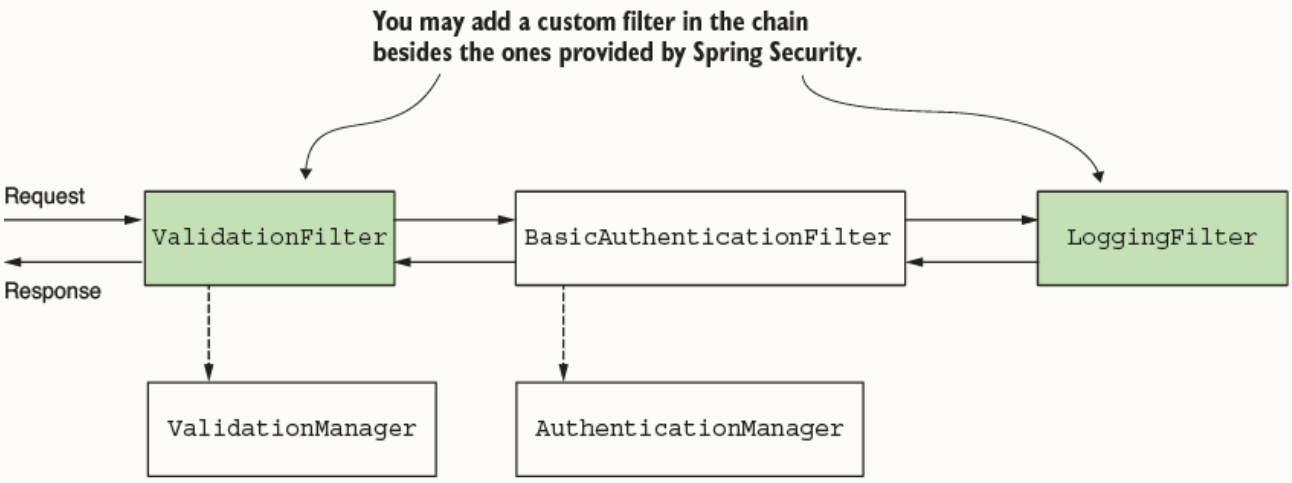


Figure 5.3 You have the option to personalize the filter chain by inserting new filters preceding, following, or in place of current ones. By doing this, you can tailor not only the authentication process but also the complete treatment of requests and responses.

5.1 Implementing filters in the Spring Security architecture

This section discusses the way filters and the filter chain work in Spring Security architecture. You need this general overview first to understand the implementation examples we'll work on in the sections that follow. In chapters 2 and 3, we learned that the authentication filter intercepts the request and delegates authentication responsibility further to the authorization manager. If we want to execute certain logic before authentication, we do this by inserting a filter before the authentication filter.

The filters in Spring Security architecture are typical HTTP filters. We can create filters by implementing the `Filter` interface from the jakarta.servlet package. As for any other HTTP filter, you need to override the `doFilter()` method to implement its logic. This method receives the `ServletRequest`, `ServletResponse`, and `FilterChain` as parameters:

- `ServletRequest`—Represents the HTTP request. We use the `ServletRequest` object to retrieve details about the request.
- `ServletResponse`—Represents the HTTP response. We use the `ServletResponse` object to alter the response before sending it back to the client or further along the filter chain.

- `FilterChain` —Represents the chain of filters. We use the `FilterChain` object to forward the request to the next filter in the chain.

NOTE Starting with Spring Boot 3, Jakarta EE replaces the old Java EE specification. Due to this change, you'll observe that some packages changed their prefix from "javax" to "jakarta." For example, types such as `Filter`, `ServletRequest`, and `ServletResponse` were previously in the javax.servlet package, but you now find them in the jakarta.servlet package.

The *filter chain* represents a collection of filters with a defined order in which they act. Spring Security provides some filter implementations and their order for us. The following are among the provided filters:

- `BasicAuthenticationFilter` takes care of HTTP Basic authentication, if present.
- `CsrfFilter` takes care of cross-site request forgery (CSRF) protection, which we'll discuss in chapter 9.
- `CorsFilter` takes care of cross-origin resource sharing (CORS) authorization rules, which we'll also discuss in chapter 10.

You don't need to know all the filters, as you probably won't touch these directly from your code, but you do need to understand how the filter chain works and be aware of a few implementations. In this book, I only explain the filters that are essential to the various topics we discuss.

It is important to understand that an application doesn't necessarily have instances of all these filters in the chain. The chain is longer or shorter, depending on how you configure the application. For example, in chapters 2 and 3, you learned that you need to call the `httpBasic()` method of the `HttpSecurity` class if you want to use the HTTP Basic authentication method. What happens is that if you call the `httpBasic()` method, an instance of `BasicAuthenticationFilter` is added to the chain. Similarly, depending on the configurations you write, the definition of the filter chain will be affected.

You add a new filter to the chain relative to another one (figure 5.4). Or you can add a filter either before, after, or at the position of a known filter. Each position is, in fact, an index (a number), and you might find it also referred to as "the order."

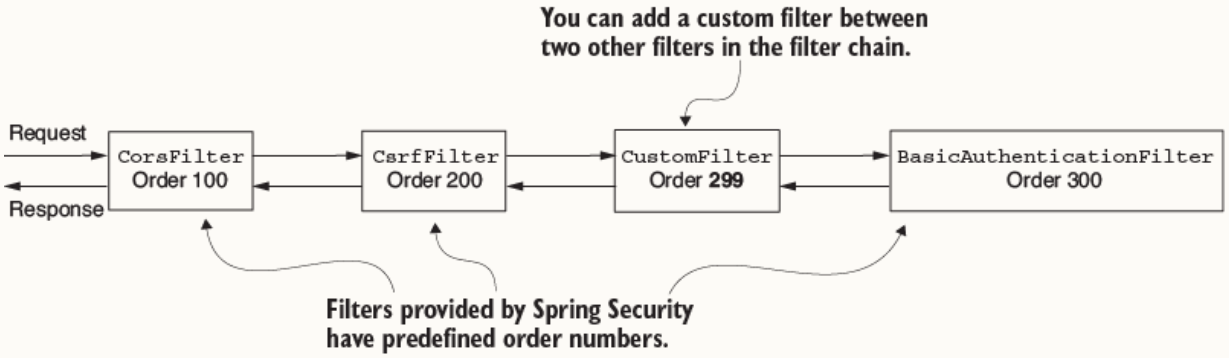


Figure 5.4 Each filter has an order number, which determines the order in which filters are applied to a request. You can add custom filters along with the filters provided by Spring Security.

If you want to learn more about the filters Spring Security provides and their configuration order, you can take a look in enum `SecurityWebFiltersOrder`, available at http://mng.bz/yZEG.

You can add two or more filters in the same position (figure 5.5). In section 5.4, we'll encounter a common case in which this might occur, one that usually creates confusion among developers.

NOTE If multiple filters have the same position, the order in which they are called is not defined.

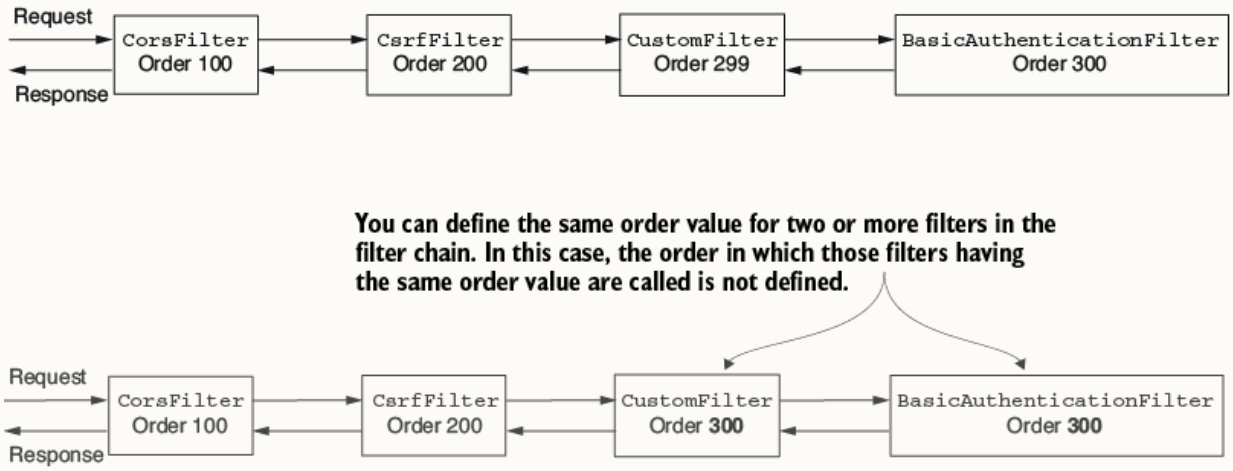


Figure 5.5 You might have multiple filters with the same order value in the chain. In this case, Spring Security doesn't guarantee the order in which they are called.

5.2 Adding a filter before an existing one in the chain

This section discusses applying custom HTTP filters before an existing one in the filter chain. You might encounter scenarios in which this is useful. To approach this problem in a practical way, we'll work on a project as an example, and you'll learn to easily implement a custom filter and apply it before an existing one in the filter chain. You can then adapt this example to any similar requirement you might find in a production application.

For our first custom filter implementation, let's consider a trivial scenario. We want to ensure that every request has a header called `Request-Id` (see project ssia-ch5-ex1). We assume that our application uses this header for tracking requests and that this header is mandatory. Simultaneously, we want to validate these assumptions before the application performs authentication. The authentication process might involve querying the database or other resource-consuming actions that we don't want the application to execute if the request format isn't valid. How do we do this? To solve the current requirement only takes two steps, and in the end, the filter chain looks like the one in figure 5.6:

1. *Implement the filter*. Create a `RequestValidationFilter` class that checks that the needed header exists in the request.
2. *Add the filter to the filter chain*. Do this in the configuration class, using the `SecurityFilterChain` bean.

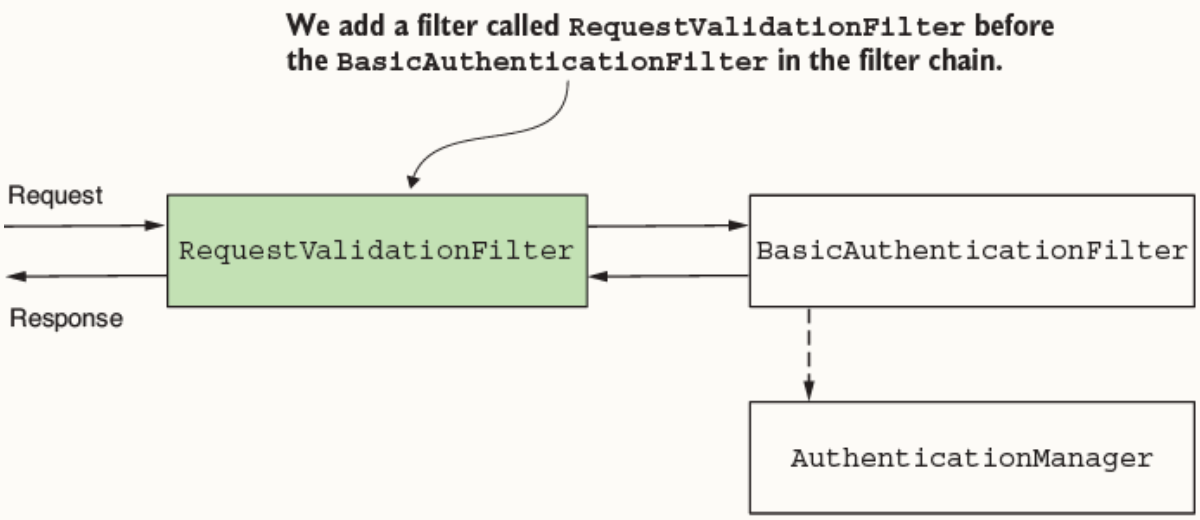


Figure 5.6 For our example, we add a `RequestValidationFilter`, which acts before the authentication filter. The `RequestValidationFilter` ensures that authentication doesn't happen if the validation of the request fails. In our case, the request must have a mandatory header named `Request-Id`.

To accomplish step 1—implementing the filter—we define a custom filter. The next listing shows the implementation.

Listing 5.1 Implementing a custom filter

```
public class RequestValidationFilter
  implements Filter {                ①

  @Override
  public void doFilter(
    ServletRequest servletRequest,
    ServletResponse servletResponse,
```

```
    FilterChain filterChain)
    throws IOException, ServletException {
    // ...
  }
}
```

① To define a filter, this class implements the Filter interface and overrides the doFilter() method.

Inside the `doFilter()` method, we write the logic of the filter. In our example, we check whether the `Request-Id` header exists. If it does, we forward the request to the next filter in the chain by calling the `doFilter()` method. If the header doesn't exist, we set an HTTP status 400 Bad Request on the response without forwarding it to the next filter in the chain (figure 5.7). Listing 5.2 presents the logic.

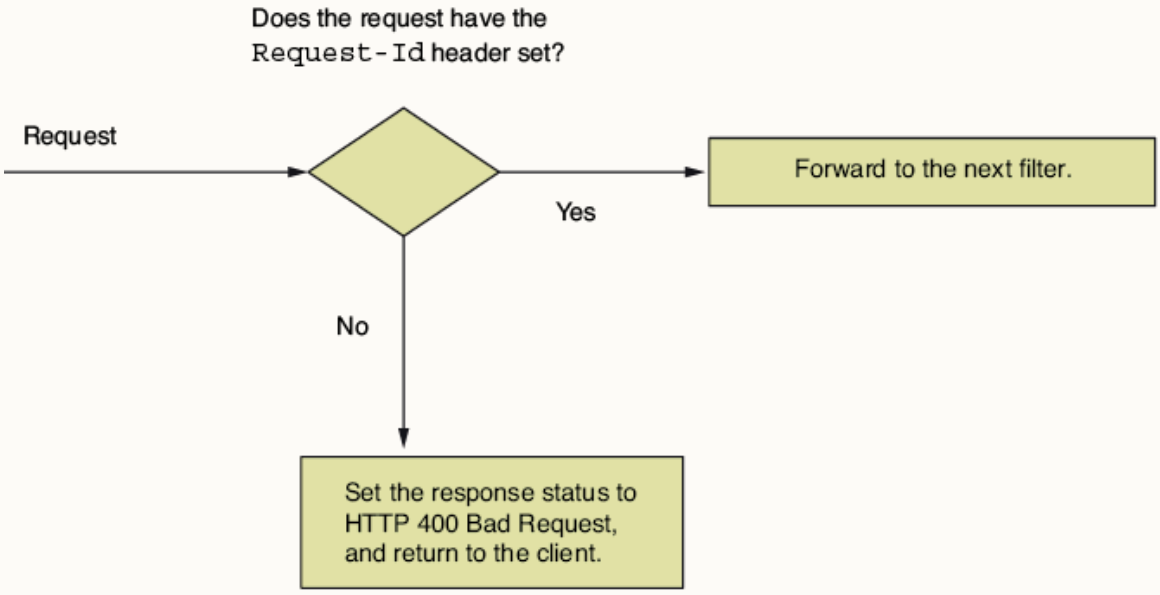


Figure 5.7 The custom filter we add before authentication checks whether the `Request-Id` header exists. If the header exists upon the request, the application forwards the request to be authenticated. If the header doesn't exist, the application sets the HTTP status 400 Bad Request and returns to the client.

Listing 5.2 Implementing the logic in the `doFilter()` method

```
@Override
public void doFilter(
  ServletRequest request,
  ServletResponse response,
  FilterChain filterChain)
    throws IOException,
        ServletException {
  var httpRequest = (HttpServletRequest) request;
  var httpResponse = (HttpServletResponse) response;
```

```
    String requestId = httpRequest.getHeader("Request-Id");

    if (requestId == null || requestId.isBlank()) {
        httpResponse.setStatus(HttpServletResponse.SC_BAD_REQUEST);
        return;                                          ①
    }

    filterChain.doFilter(request, response);             ②

}
```

① If the header is missing, the HTTP status changes to 400 Bad Request, and the request is not forwarded to the next

② If the header exists, the request is forwarded to the next filter in the chain. filter in the chain.

To implement step 2, applying the filter within the configuration class, we use the `addFilterBefore()` method of the `HttpSecurity` object because we want the application to execute this custom filter before authentication. This method receives two parameters:

- *An instance of the custom filter we want to add to the chain*—In our example, this is an instance of the `RequestValidationFilter` class presented in listing 5.1.
- *The type of filter before which we add the new instance*—For this example, because the requirement is to execute the filter logic before authentication, we need to add our custom filter instance before the authentication filter. The class `BasicAuthenticationFilter` defines the default type of the authentication filter.

Until now, we have referred to the filter dealing with authentication generally as the *authentication* filter. You'll find out in the next chapters that Spring Security also configures other filters. In chapter 9, we'll discuss cross-site request forgery (CSRF) protection, and in chapter 10, we'll discuss cross-origin resource sharing (CORS). Both capabilities also rely on filters.

The next listing shows how to add the custom filter before the authentication filter in the configuration class. To make the example simpler, we use the `permitAll()` method to allow all unauthenticated requests.

Listing 5.3 Configuring the custom filter before authentication

```
@Configuration
public class ProjectConfig {

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.addFilterBefore(                    ①
      new RequestValidationFilter(), BasicAuthenticationFilter.class)
      .authorizeRequests(c -> c.anyRequest().permitAll());

    return http.build();
  }
}
```

① Adds an instance of the custom filter before the authentication filter in the filter chain

We also need a controller class and an endpoint to test the functionality. The next listing defines the controller class.

Listing 5.4 The controller class

```
@RestController
public class HelloController {

  @GetMapping("/hello")
  public String hello() {
    return "Hello!";
  }
}
```

You can now run and test the application. Calling the endpoint without the header generates a response with HTTP status 400 Bad Request. If you add the header to the request, the response status becomes HTTP 200 OK, and you'll also see the response body, `Hello!` To call the endpoint without the `Request-Id` header, we use this cURL command:

```
curl -v http://localhost:8080/hello
```

This call generates the following (truncated) response:

```
...
< HTTP/1.1 400
...
```

To call the endpoint and provide the `Request-Id` header, we use this cURL command:

```
curl -H "Request-Id:12345" http://localhost:8080/hello
```

This call generates the following response body:

```
Hello!
```

5.3 Adding a filter after an existing one in the chain

This section illustrates how to add a filter after an existing one in the filter chain. This approach is used when you want to execute some logic after something already existing in the filter chain. Let's assume that you must execute some logic after the authentication process. Examples of this could be notifying a different system after certain authentication events or simply for logging and tracing purposes (figure 5.8). Like in section 5.1, we implement an example to show how to do this. You can adapt it to your needs for a real-world scenario.

For our example, we log all successful authentication events by adding a filter after the authentication filter (figure 5.8). We consider that what bypasses the authentication filter represents a successfully authenticated event, and we want to log it. Continuing the example from section 5.1, we also log the request ID received through the HTTP header.

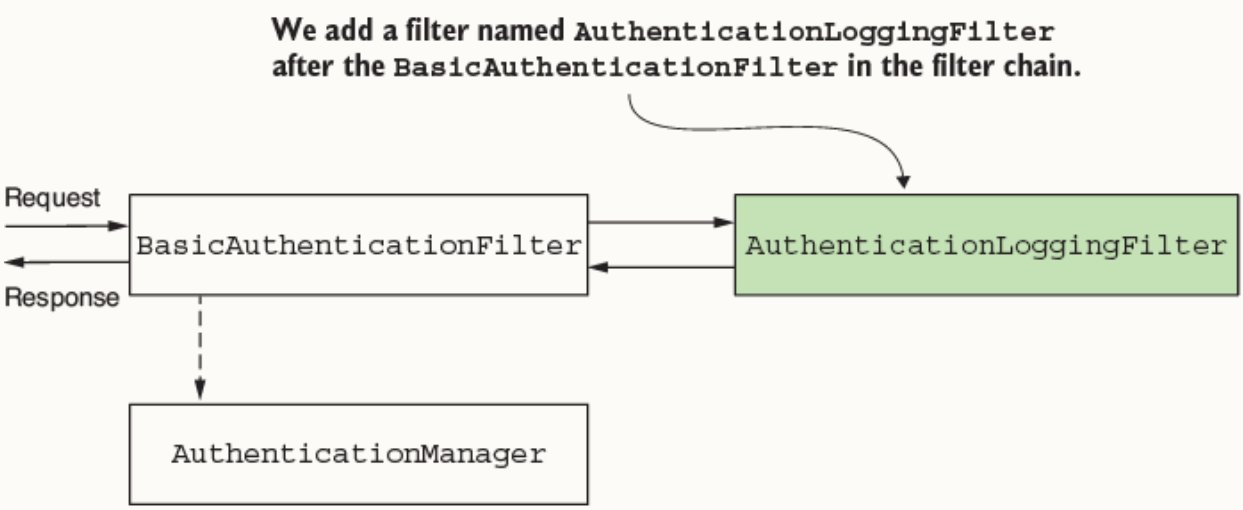


Figure 5.8 We add the `AuthenticationLoggingFilter` after the `BasicAuthenticationFilter` to log the requests that the application authenticates.

The following listing presents the definition of a filter that logs requests that pass the authentication filter.

Listing 5.5 Defining a filter to log requests

```
public class AuthenticationLoggingFilter implements Filter {

  private final Logger logger =
        Logger.getLogger(
        AuthenticationLoggingFilter.class.getName());

  @Override
  public void doFilter(
    ServletRequest request,
    ServletResponse response,
    FilterChain filterChain)
      throws IOException, ServletException {

      var httpRequest = (HttpServletRequest) request;

      var requestId =
        httpRequest.getHeader("Request-Id");          ①

      logger.info("Successfully authenticated         ②
              request with id " +  requestId);     ②

      filterChain.doFilter(request, response);        ③
  }
}
```

① Gets the request ID from the request headers

② Logs the event with the value of the request ID

③ Forwards the request to the next filter in the chain

To add the custom filter in the chain after the authentication filter, you call the `addFilterAfter()` method of `HttpSecurity`. The next listing shows the implementation.

Listing 5.6 Adding a custom filter after an existing one in the filter chain

```
@Configuration
public class ProjectConfig {

 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity http)
  throws Exception {

  http.addFilterBefore(
        new RequestValidationFilter(),
        BasicAuthenticationFilter.class)
      .addFilterAfter(                    ①
        new AuthenticationLoggingFilter(),
        BasicAuthenticationFilter.class)
      .authorizeRequests(c -> c.anyRequest().permitAll());

  return http.build();
 }
}
```

① Adds an instance of AuthenticationLoggingFilter to the filter chain after the authentication filter

After running the application and calling the endpoint, we observe that for every successful call to the endpoint, the application prints a log line in the console. For the call

```
curl -H "Request-Id:12345" http://localhost:8080/hello
```

the response body is

```
Hello!
```

In the console, you can see a line similar to

```
INFO 5876 --- [nio-8080-exec-2]
[CA]c.l.s.f.AuthenticationLoggingFilter:
[CA]Successfully authenticated request with id 12345
```

5.4 Adding a filter at the location of another in the chain

This section discusses adding a filter at the location of another one in the filter chain. This approach can especially be used when providing a different implementation for a responsibility that is already assumed by one of the filters known by Spring Security. A typical scenario is authentication.

Let's assume that instead of the HTTP Basic authentication flow, you want to implement something different. Instead of using a username and a password as input credentials based on which the application authenticates the user, you need to apply another approach. Some examples of scenarios that you could encounter are

- Identification based on a static header value for authentication
- Using a symmetric key to sign the request for authentication
- Using a one-time password (OTP) in the authentication process

In our first scenario (identification based on a static key for authentication), the client sends a string to the app in the header of the HTTP request, which is always the same. The application stores these values somewhere, most likely in a database or a secrets vault. Based on this static value, the application identifies the client.

This approach (figure 5.9) offers weak security related to authentication, but architects and developers often choose it in calls between backend applications for its simplicity. The implementations also execute fast because these don't need to do complex calculations, like in the case of applying a cryptographic signature. This way, static keys used for authentication represent a compromise where developers rely more on the infrastructure level in terms of security and also don't leave the endpoints wholly unprotected.

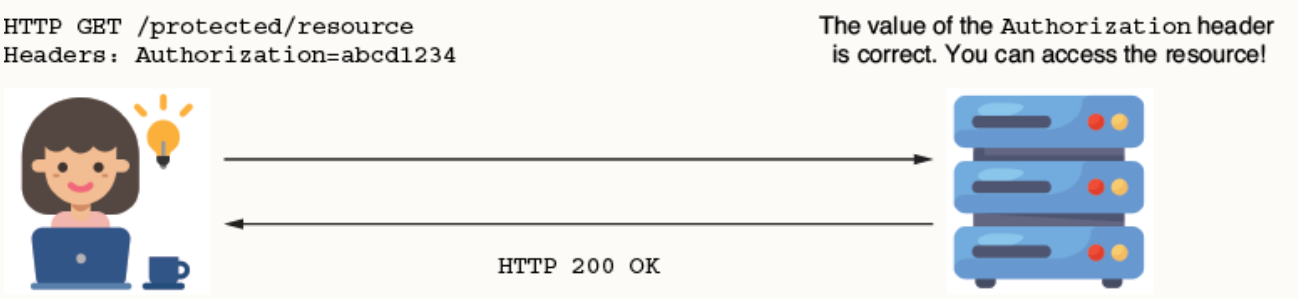


Figure 5.9 The request contains a header with the value of the static key. If this value matches the one known by the application, it accepts the request.

In our second scenario, using symmetric keys to sign and validate requests, both client and server know the value of a key (client and server share the key). The client uses this key to sign a part of the request (for example, to sign the value of specific headers), and the server checks whether the signature is valid using the same key (figure 5.10). The server can store individual keys for each client in a database or a secrets vault. Similarly, you can use a pair of asymmetric keys.

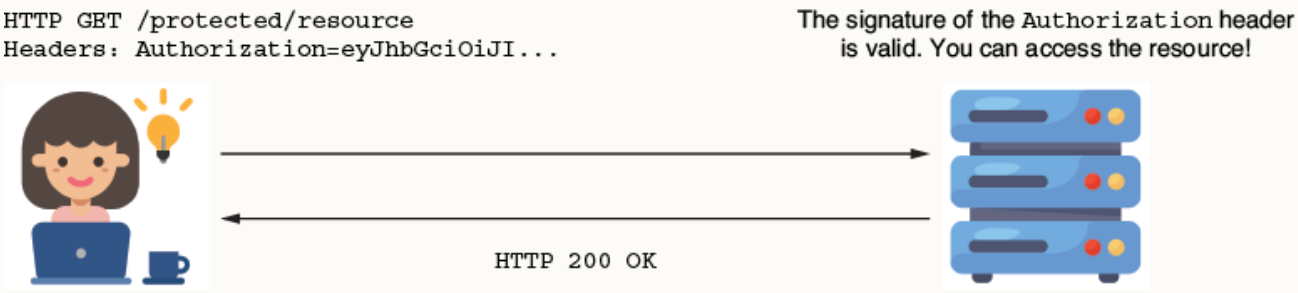


Figure 5.10 The `Authorization` header holds a value encrypted with a key shared between the client and server (or encrypted with a private key for which the server possesses the corresponding public key). If the application verifies the signature is valid, it permits the request to proceed.

Finally, for our third scenario, using an OTP in the authentication process, the user receives the OTP via a message or by using an authentication provider app such as Google Authenticator (figure 5.11).

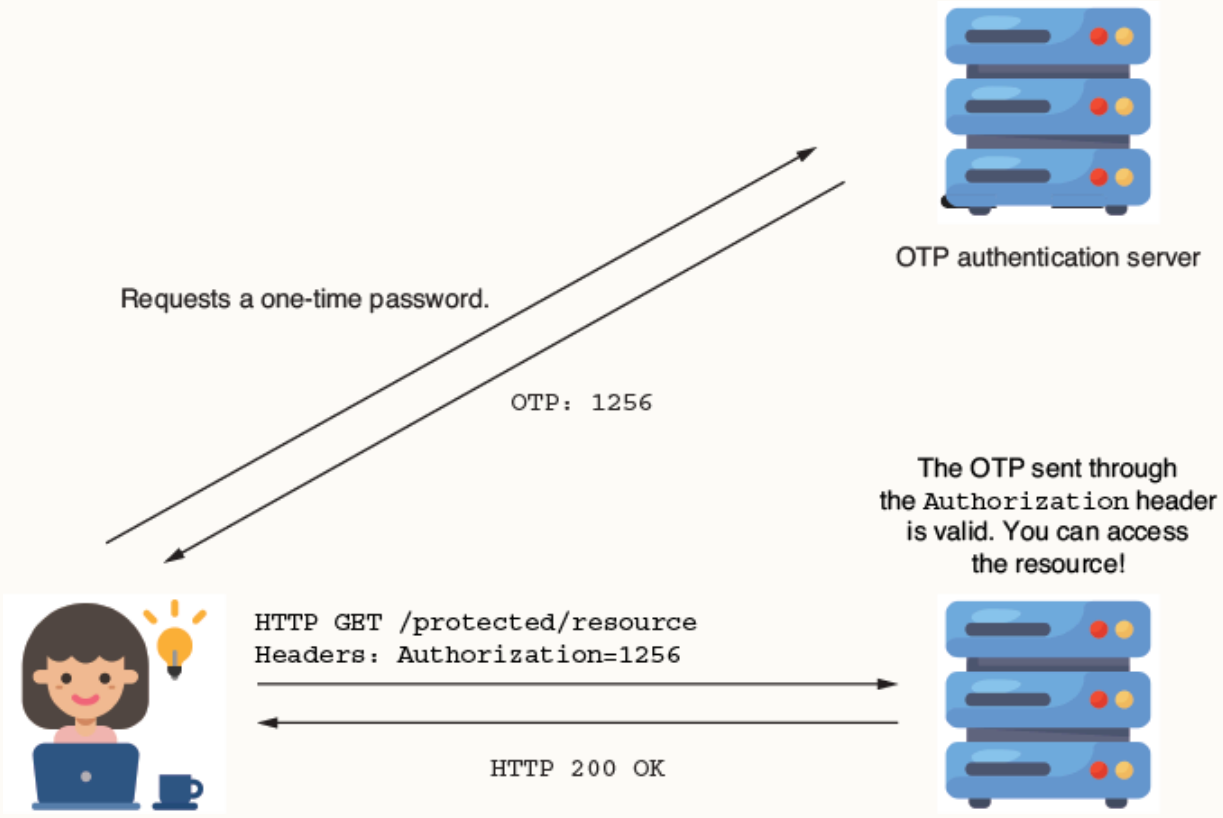


Figure 5.11 For resource entry, the client must utilize a one-time password (OTP). This OTP is acquired from an external authentication server. Typically, applications employ this method for login processes that necessitate multi-factor authentication.

Let's implement an example to demonstrate how to apply a custom filter. To keep the case relevant but straightforward, we focus on configuration and consider a simple logic for authentication. In our scenario, we have the value of a static key, which is the same for all requests. To be authenticated, the user must add the correct value of the static key in the `Authorization` header, as presented in figure 5.12. You can find the code for this example in the project ssia-ch5-ex2.

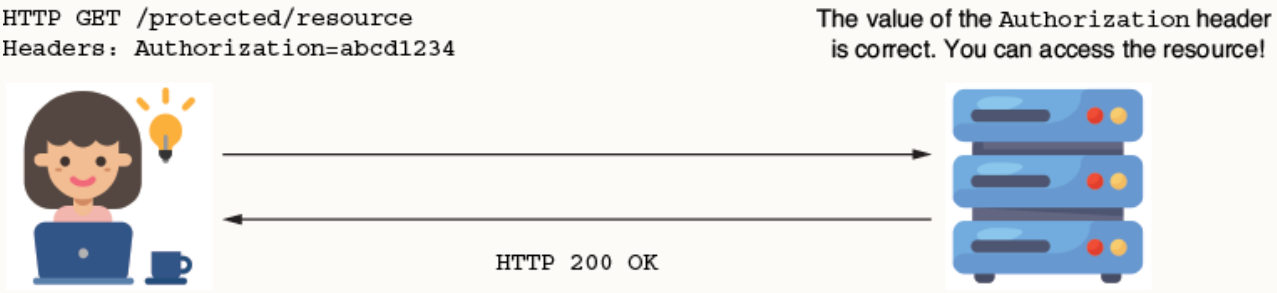


Figure 5.12 The client adds a static key in the `Authorization` header of the HTTP request. The server checks whether it knows the key before authorizing the requests.

We start with implementing the filter class, named `StaticKeyAuthentication-Filter`. This class reads the value of the static key from the properties file and verifies whether the value of the `Authorization` header is equal to it. If the values are the same, the filter forwards the request to the next component in the filter chain. If not, the filter sets the value 401 Unauthorized to the HTTP status of the response without forwarding the re-

quest in the filter chain. The following listing defines the `StaticKeyAuthenticationFilter` class.

Listing 5.7 The definition of the `StaticKeyAuthenticationFilter` class

```
@Component                                    ①
public class StaticKeyAuthenticationFilter
 implements Filter {                          ②

 @Value("${authorization.key}")               ③
 private String authorizationKey;

 @Override
 public void doFilter(ServletRequest request,
                      ServletResponse response,
                      FilterChain filterChain)
  throws IOException, ServletException {

  var httpRequest = (HttpServletRequest) request;
  var httpResponse = (HttpServletResponse) response;

  String authentication =                     ④
       httpRequest.getHeader("Authorization");

  if (authorizationKey.equals(authentication)) {
     filterChain.doFilter(request, response);
  } else {
     httpResponse.setStatus(
        HttpServletResponse.SC_UNAUTHORIZED);
  }
 }
}
```

① To allow us to inject values from the properties file, it adds an instance of the class in the Spring context.

② Defines the authentication logic by implementing the Filter interface and overriding the doFilter() method

③ Takes the value of the static key from the properties file using the @Value annotation

④ Takes the value of the Authorization header from the request to compare it with the static key

Once we define the filter, we add it to the filter chain at the position of the class `BasicAuthenticationFilter` by using the `addFilterAt()` method (figure 5.13).

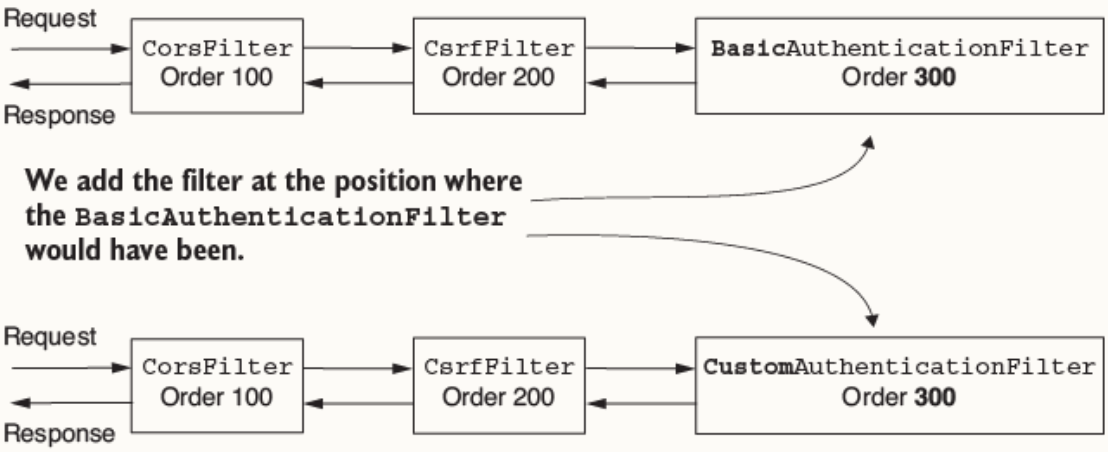


Figure 5.13 We add our custom authentication filter at the location where the class `BasicAuthenticationFilter` would have been if we were using HTTP Basic as an authentication method. This means our custom filter has the same ordering value.

But remember what we discussed in section 5.1. When adding a filter at a specific position, Spring Security does not assume it is the only filter at that position. You might add more filters at the same location in the chain. In this case, Spring Security doesn't guarantee the order in which these will act. I repeat this because I've seen many people confused by how it works. Some developers think that when you apply a filter at a position of a known one, it will be replaced. This is not the case! We must make sure we do not add filters that we don't need.

NOTE I advise you not to add multiple filters at the same position in the chain. When you add more filters to the same location, the order in which they are used is not defined. It makes sense to have a definite order in which filters are called. Having a known order makes your application easier to understand and maintain.

In listing 5.8, you can find the definition of the configuration class that adds the filter. Observe that we don't call the `httpBasic()` method from the `HttpSecurity` class here because we don't want the `BasicAuthenticationFilter` instance to be added to the filter chain.

Listing 5.8 Adding the filter in the configuration class

```
@Configuration
public class ProjectConfig {

  private final StaticKeyAuthenticationFilter filter;      ①
```

```
  // omitted constructor

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {
    http.addFilterAt(filter,                    ②
        BasicAuthenticationFilter.class)
      .authorizeRequests(c -> c.anyRequest().permitAll());

    return http.build();
  }
}
```

① Injects the instance of the filter from the Spring context

② Adds the filter at the position of the basic authentication filter in the filter chain

To test the application, we also need an endpoint. For that, we define a controller, like in listing 5.4. You should add a value for the static key on the server in the application.properties file, as shown in

```
authorization.key=SD9cICjl1e
```

NOTE Storing passwords, keys, or any other data not meant to be seen by everybody in the properties file is never a good idea for a production application. In our examples, we use this approach for simplicity and to let you focus on the Spring Security configurations we make. But in real-world scenarios, make sure to use a secrets vault to store such kinds of details.

We can now test the application. It is expected that the app will allow requests having the correct value for the `Authorization` header and reject others, returning an HTTP 401 Unauthorized status on the response. The code snippets that follow present the `curl` calls used to test the application. If you use the same value set on the server side for the `Authorization` header, the call is successful, and you'll see the response body, `Hello!` The call

```
curl -H "Authorization:SD9cICjl1e" http://localhost:8080/hello
```

returns this response body:

```
Hello!
```

With the following call, if the `Authorization` header is missing or is incorrect, the response status is HTTP 401 Unauthorized:

```
curl -v http://localhost:8080/hello
```

The response status is

```
...
< HTTP/1.1 401
...
```

In this case, because we don't configure a `UserDetailsService`, Spring Boot configures one automatically, as you learned in chapter 2. But in our scenario, you don't need a `UserDetailsService` at all because the concept of the user doesn't exist. We only validate that the user requesting to call an endpoint on the server knows a given value. Application scenarios are not usually this simple, and they often require a `UserDetailsService`. However, if you anticipate or have a case where this component is not needed, you can disable autoconfiguration. To disable the configuration of the default `UserDetailsService`, you can use the `exclude` attribute of the `@SpringBootApplication` annotation on the main class:

```
@SpringBootApplication(exclude =
 {UserDetailsServiceAutoConfiguration.class })
```

5.5 Filter implementations provided by Spring Security

This section discusses classes provided by Spring Security that implement the `Filter` interface. In the examples, we define the filter by implementing this interface directly.

Spring Security offers a few abstract classes that implement the `Filter` interface and for which you can extend your filter definitions. These classes also add functionality your implementations could benefit from when you extend them. For example, you could extend the `GenericFilterBean` class, which allows you to use initialization parameters that you would define in a web.xml descriptor file where applicable. A more useful class that extends the `GenericFilterBean` is `OncePerRequestFilter`. When adding a filter to the

chain, the framework doesn't guarantee it will be called only once per request. `OncePerRequestFilter`, as the name suggests, implements logic to make sure that the filter's `doFilter()` method is executed only one time per request.

If you need such functionality in your application, use the classes that Spring provides. However, if you don't need them, I'd always recommend going as simple as possible with your implementations. Too often, I've seen developers extending the `GenericFilterBean` class instead of implementing the `Filter` interface in functionalities that don't require the custom logic added by the `GenericFilterBean` class. When asked why, it seems they don't know. They probably copied the implementation as they found it in examples on the web.

To make it crystal clear how to use such a class, let's write an example. The logging functionality we implemented in section 5.3 is a great candidate for using `OncePerRequestFilter`. We want to avoid logging the same requests multiple times. Spring Security doesn't guarantee the filter won't be called more than once, so we have to take care of this ourselves. The easiest way is to implement the filter using the `OncePerRequestFilter` class. I wrote this in a separate project called ssia-ch5-ex3.

In listing 5.9, you'll find the change I made for the `AuthenticationLoggingFilter` class. Instead of implementing the `Filter` interface directly, as was the case in the example in section 5.3, it now extends the `OncePerRequestFilter` class. The method we override here is `doFilterInternal()`. You can find this code in project ssia-ch5-ex3.

Listing 5.9 Extending the `OncePerRequestFilter` class

```
public class AuthenticationLoggingFilter
  extends OncePerRequestFilter {                    ①

  private final Logger logger =
        Logger.getLogger(
          AuthenticationLoggingFilter.class.getName());

  @Override
  protected void doFilterInternal(                  ②
    HttpServletRequest request,                     ③
    HttpServletResponse response,                   ③
    FilterChain filterChain) throws
      ServletException, IOException {

      String requestId = request.getHeader("Request-Id");
```

```
        logger.info("Successfully authenticated request with id " +
                requestId);

        filterChain.doFilter(request, response);
    }
}
```

① Instead of implementing the Filter interface, it extends the OncePerRequestFilter class.

② Overrides doFilterInternal(), which replaces the purpose of the doFilter() method of the Filter interface

③ The OncePerRequestFilter only supports HTTP filters. This is why the parameters are directly given as HttpServletRequest and HttpServletResponse.

A few short observations about the `OncePerRequestFilter` class that you might find useful:

- *It supports only HTTP requests, but that's actually what we always use.* The advantage is that it casts the types, and we directly receive the requests as `HttpServletRequest` and `HttpServletResponse`. Remember, with the `Filter` interface, we had to cast the request and the response.
- *You can implement logic to decide whether the filter is applied.* Even if you added the filter to the chain, you might decide it doesn't apply to certain requests. You set this by overriding the `shouldNotFilter(HttpServletRequest)` method. By default, the filter applies to all requests.
- *By default, a* `OncePerRequestFilter` *doesn't apply to asynchronous requests or error dispatch requests.* You can change this behavior by overriding the methods `shouldNotFilterAsyncDispatch()` and `shouldNotFilterErrorDispatch()`.

If you find any of these characteristics of the `OncePerRequestFilter` useful in your implementation, I recommend using this class to define your filters.

Summary

- The first layer of the web application architecture, which intercepts HTTP requests, is a filter chain. As for other components in Spring Security architecture, you can customize them to match your requirements.
- You can customize the filter chain by adding new filters before, after, or at the position of an existing filter.

- You can have multiple filters at the same position of an existing filter. In this case, the order in which the filters are executed is not defined.
- Changing the filter chain helps you customize authentication and authorization to match the requirements of your application.

Previous chapter
< 4 Managing passwords

Next chapter
6 Implementing authentications >